

# Curso Especial de Programación I: Punteros, new y delete

## Objetivo

Practicar el uso básico de punteros, memoria dinámica con `new` y `delete`, arreglos dinámicos con `new[]` y `delete[]`, funciones que reciben punteros y funciones `void` que modifican datos del programa principal.

## Indicaciones generales

El alumno deberá entregar un documento en PDF o Word con evidencia de la realización de los ejercicios.

El documento debe incluir:

- Portada con nombre completo, grupo y fecha.
- Código fuente de cada ejercicio.
- Captura de pantalla de la ejecución de cada ejercicio.
- Una breve explicación de lo que hace cada programa.

## Restricciones

En esta tarea no se permite usar:

- `std::vector`
- Variables globales para resolver los cálculos principales
- Programación orientada a objetos
- `malloc`
- `calloc`
- `realloc`
- `free`

Para esta tarea se debe usar:

- Punteros
- new
- delete
- new[]
- delete[]
- Funciones

## Ejercicio 1: Variable dinámica

### Descripción

Crear un programa que reserve memoria dinámica para una sola variable de tipo `float`. Esa variable representará una calificación.

El programa debe:

1. Declarar un puntero de tipo `float`.
2. Reservar memoria con `new`.
3. Capturar una calificación entre 0 y 10.
4. Mostrar la calificación.
5. Mostrar la dirección guardada en el puntero.
6. Liberar memoria con `delete`.
7. Asignar `nullptr` al puntero después de liberar.

### Código base sugerido

```
1  /*
2  Ejercicio 1
3  Variable dinamica con new y delete
4  */
5
6  #include <iostream>
7  using namespace std;
8
9  bool calificacionValida(float calificacion);
10
11 int main() {
12 float *calificacion = nullptr;
13
14 ...
15 // Reservar memoria dinamica aqui.
16
```

```

17 // Capturar y validar la calificacion aqui.
18
19 // Mostrar el valor y la direccion aqui.
20
21 // Liberar memoria aqui.
22
23 return 0;
24 ...
25
26 }
27
28 bool calificacionValida(float calificacion) {
29 return calificacion >= 0 && calificacion <= 10;
30 }

```

## Ejercicio 2: Arreglo dinámico de calificaciones

### Descripción

Crear un programa que permita capturar una cantidad variable de calificaciones usando un arreglo dinámico.

El programa debe:

1. Pedir al usuario cuántas calificaciones desea capturar.
2. Reservar memoria con `new[]`.
3. Capturar las calificaciones.
4. Mostrar las calificaciones.
5. Calcular suma, promedio, mayor y menor usando una función `void`.
6. Liberar memoria con `delete[]`.

### Funciones mínimas requeridas

```

1 int pedirCantidad();
2 bool calificacionValida(float calificacion);
3 void capturarCalificaciones(float *calificaciones, int n);
4 void mostrarCalificaciones(float *calificaciones, int n);
5 void calcularResumen(float *calificaciones, int n,
6 float *suma,
7 float *promedio,
8 float *mayor,
9 float *menor);

```

## Requisito importante

La función `calcularResumen` debe ser de tipo `void` y debe modificar las variables del `main` mediante punteros.

Ejemplo de llamada:

```
1 calcularResumen(calificaciones, n, &suma, &promedio, &mayor, &menor);
```

## Ejercicio 3: Arreglo dinámico de productos

### Descripción

Crear un programa que use memoria dinámica para capturar productos. Cada producto debe tener:

- Código
- Nombre
- Precio
- Existencia

El programa debe:

1. Pedir cuántos productos se desean capturar.
2. Reservar un arreglo dinámico de estructuras usando `new[]`.
3. Capturar los productos.
4. Mostrar los productos.
5. Calcular el valor total del inventario.
6. Liberar memoria con `delete[]`.

### Estructura requerida

```
1 typedef struct {  
2     int codigo;  
3     char nombre[40];  
4     float precio;  
5     int existencia;  
6 } Producto;
```

## Funciones mínimas requeridas

```
1 int pedirCantidadProductos();
2 void capturarProductos(Producto *productos, int n);
3 void mostrarProductos(Producto *productos, int n);
4 float calcularValorInventario(Producto *productos, int n);
```

## Ejercicio 4: Ordenamiento usando punteros

### Descripción

Agregar al ejercicio 3 una función que ordene los productos por código de menor a mayor. La función debe ser de tipo void.

### Funciones requeridas

```
1 void intercambiarProductos(Producto *a, Producto *b);
2 void ordenarPorCodigo(Producto *productos, int n);
```

### Requisito

La función `ordenarPorCodigo` debe modificar directamente el arreglo dinámico original. No debe regresar otro arreglo.

## Forma de entrega

El alumno debe entregar un archivo con:

- Código del ejercicio 1.
- Código del ejercicio 2.
- Código del ejercicio 3.
- Código del ejercicio 4.

## Rúbrica

| Criterio                                                                                  | Puntos |
|-------------------------------------------------------------------------------------------|--------|
| Ejercicio 1 funcionando correctamente                                                     | 15     |
| Ejercicio 2 usando <code>new[]</code> , <code>delete[]</code> y función void con punteros | 25     |
| Ejercicio 3 usando arreglo dinámico de estructuras                                        | 25     |
| Ejercicio 4 ordenando productos mediante punteros                                         | 20     |

|                                      |            |
|--------------------------------------|------------|
| Evidencia completa en PDF o Word     | 10         |
| Código ordenado, comentado y legible | 5          |
| <b>Total</b>                         | <b>100</b> |

## Notas finales

Se revisará que el programa libere correctamente la memoria reservada.

Recordatorio:

```

1  int *datos = new int[n];
2
3  // usar datos
4
5  delete[] datos;
6  datos = nullptr;

```

Y para una sola variable:

```

1  int *numero = new int;
2
3  // usar numero
4
5  delete numero;
6  numero = nullptr;

```